

UNIT 3

AIML

What is Machine Learning?

In the real world, we are surrounded by humans who can learn everything from their experiences with their learning capability, and we have computers or machines which work on our instructions. But can a machine also learn from experiences or past data like a human does? So here comes the role of **Machine Learning**.

Introduction to Machine Learning

A subset of artificial intelligence known as machine learning focuses primarily on the creation of algorithms that enable a computer to independently learn from data and previous experiences. Arthur Samuel first used the term "machine learning" in 1959. It could be summarized as follows:

Without being explicitly programmed, machine learning enables a machine to automatically learn from data, improve performance from experiences, and predict things.

Machine learning algorithms create a mathematical model that, without being explicitly programmed, aids in making predictions or decisions with the assistance of sample historical data, or training data. For the purpose of developing predictive models, machine learning brings together statistics and computer science. Algorithms that learn from historical data are either constructed or utilized in machine learning. The performance will rise in proportion to the quantity of information we provide.

A machine can learn if it can gain more data to improve its performance.

How does Machine Learning work

A machine learning system builds prediction models, learns from previous data, and predicts the output of new data whenever it receives it. The amount of data helps to build a better model that accurately predicts the output, which in turn affects the accuracy of the predicted output.

Let's say we have a complex problem in which we need to make predictions. Instead of writing code, we just need to feed the data to generic algorithms, which build the logic based on the data and predict the output. Our perspective on the issue has changed as a result of machine learning. The Machine Learning algorithm's operation is depicted in the following block diagram:

Features of Machine Learning:

- Machine learning uses data to detect various patterns in a given dataset.
- It can learn from past data and improve automatically.
- It is a data-driven technology.
- Machine learning is much similar to data mining as it also deals with the huge amount of the data.

Need for Machine Learning

The demand for machine learning is steadily rising. Because it is able to perform tasks that are too complex for a person to directly implement, machine learning is required. Humans are constrained by our inability to manually access vast amounts of data; as a result, we require computer systems, which is where machine learning comes in to simplify our lives.

By providing them with a large amount of data and allowing them to automatically explore the data, build models, and predict the required output, we can train machine learning algorithms. The cost function can be used to determine the amount of data and the machine learning algorithm's performance. We can save both time and money by using machine learning.

The significance of AI can be handily perceived by its utilization's cases, Presently, AI is utilized in self-driving vehicles, digital misrepresentation identification, face acknowledgment, and companion idea by Facebook, and so on. Different top organizations, for example, Netflix and Amazon have constructed AI models that are utilizing an immense measure of information to examine the client interest and suggest item likewise.

Following are some key points which show the importance of Machine Learning:

- Rapid increment in the production of data
- Solving complex problems, which are difficult for a human
- Decision making in various sector including finance
- Finding hidden patterns and extracting useful information from data.

Classification of Machine Learning

At a broad level, machine learning can be classified into three types:

1. **Supervised learning**
2. **Unsupervised learning**
3. **Reinforcement learning**

1) Supervised Learning

In supervised learning, sample labeled data are provided to the machine learning system for training, and the system then predicts the output based on the training data.

The system uses labeled data to build a model that understands the datasets and learns about each one. After the training and processing are done, we test the model with sample data to see if it can accurately predict the output.

The mapping of the input data to the output data is the objective of supervised learning. The managed learning depends on oversight, and it is equivalent to when an understudy learns things in the management of the educator. Spam filtering is an example of supervised learning.

Supervised learning can be grouped further in two categories of algorithms:

- **Classification**
- **Regression**

2) Unsupervised Learning

Unsupervised learning is a learning method in which a machine learns without any supervision.

The training is provided to the machine with the set of data that has not been labeled, classified, or categorized, and the algorithm needs to act on that data without any supervision. The goal of unsupervised learning is to restructure the input data into new features or a group of objects with similar patterns.

In unsupervised learning, we don't have a predetermined result. The machine tries to find useful insights from the huge amount of data. It can be further classified into two categories of algorithms:

- **Clustering**
- **Association**

3) Reinforcement Learning

Reinforcement learning is a feedback-based learning method, in which a learning agent gets a reward for each right action and gets a penalty for each wrong action. The agent learns automatically with these feedbacks and improves its performance. In reinforcement learning, the agent interacts with the environment and explores it. The goal of an agent is to get the most reward points, and hence, it improves its performance.

The robotic dog, which automatically learns the movement of his arms, is an example of Reinforcement learning.

Note: We will learn about the above types of machine learning in detail in later chapters.

History of Machine Learning

Before some years (about 40-50 years), machine learning was science fiction, but today it is the part of our daily life. Machine learning is making our day to day life easy from **self-driving cars** to **Amazon virtual assistant "Alexa"**. However, the idea behind machine learning is so old and has a long history. Below some milestones are given which have occurred in the history of machine learning:

The early history of Machine Learning (Pre-1940):

- **1834:** In 1834, Charles Babbage, the father of the computer, conceived a device that could be programmed with punch cards. However, the machine was never built, but all modern computers rely on its logical structure.
- **1936:** In 1936, Alan Turing gave a theory that how a machine can determine and execute a set of instructions.

The era of stored program computers:

- **1940:** In 1940, the first manually operated computer, "ENIAC" was invented, which was the first electronic general-purpose computer. After that stored program computer such as EDSAC in 1949 and EDVAC in 1951 were invented.
- **1943:** In 1943, a human neural network was modeled with an electrical circuit. In 1950, the scientists started applying their idea to work and analyzed how human neurons might work.

Computer machinery and intelligence:

- **1950:** In 1950, Alan Turing published a seminal paper, "**Computer Machinery and Intelligence**," on the topic of artificial intelligence. **In his paper, he asked, "Can machines think?"**

Machine intelligence in Games:

- **1952:** Arthur Samuel, who was the pioneer of machine learning, created a program that helped an IBM computer to play a checkers game. It performed better more it played.
- **1959:** In 1959, the term "Machine Learning" was first coined by **Arthur Samuel**.

The first "AI" winter:

- The duration of 1974 to 1980 was the tough time for AI and ML researchers, and this duration was called as **AI winter**.
- In this duration, failure of machine translation occurred, and people had reduced their interest from AI, which led to reduced funding by the government to the researches.

Machine Learning from theory to reality

- **1959:** In 1959, the first neural network was applied to a real-world problem to remove echoes over phone lines using an adaptive filter.

- **1985:** In 1985, Terry Sejnowski and Charles Rosenberg invented a neural network **NETtalk**, which was able to teach itself how to correctly pronounce 20,000 words in one week.
- **1997:** The IBM's **Deep blue** intelligent computer won the chess game against the chess expert Garry Kasparov, and it became the first computer which had beaten a human chess expert.

Machine Learning at 21st century

2006:

- Geoffrey Hinton and his group presented the idea of profound getting the hang of utilizing profound conviction organizations.
- The Elastic Compute Cloud (EC2) was launched by Amazon to provide scalable computing resources that made it easier to create and implement machine learning models.

2007:

- Participants were tasked with increasing the accuracy of Netflix's recommendation algorithm when the Netflix Prize competition began.
- Support learning made critical progress when a group of specialists utilized it to prepare a PC to play backgammon at a top-notch level.

2008:

- Google delivered the Google Forecast Programming interface, a cloud-based help that permitted designers to integrate AI into their applications.
- Confined Boltzmann Machines (RBMs), a kind of generative brain organization, acquired consideration for their capacity to demonstrate complex information conveyances.

2009:

- Profound learning gained ground as analysts showed its viability in different errands, including discourse acknowledgment and picture grouping.

- The expression "Large Information" acquired ubiquity, featuring the difficulties and open doors related with taking care of huge datasets.

2010:

- The ImageNet Huge Scope Visual Acknowledgment Challenge (ILSVRC) was presented, driving progressions in PC vision, and prompting the advancement of profound convolutional brain organizations (CNNs).

2011:

- On Jeopardy! IBM's Watson defeated human champions., demonstrating the potential of question-answering systems and natural language processing.

2012:

- AlexNet, a profound CNN created by Alex Krizhevsky, won the ILSVRC, fundamentally further developing picture order precision and laying out profound advancing as a predominant methodology in PC vision.
- Google's Cerebrum project, drove by Andrew Ng and Jeff Dignitary, utilized profound figuring out how to prepare a brain organization to perceive felines from unlabeled YouTube recordings.

2013:

- Ian Goodfellow introduced generative adversarial networks (GANs), which made it possible to create realistic synthetic data.
- Google later acquired the startup DeepMind Technologies, which focused on deep learning and artificial intelligence.

2014:

- Facebook presented the DeepFace framework, which accomplished close human precision in facial acknowledgment.
- AlphaGo, a program created by DeepMind at Google, defeated a world champion Go player and demonstrated the potential of reinforcement learning in challenging games.

2015:

- Microsoft delivered the Mental Toolbox (previously known as CNTK), an open-source profound learning library.
- The performance of sequence-to-sequence models in tasks like machine translation was enhanced by the introduction of the idea of attention mechanisms.

2016:

- The goal of explainable AI, which focuses on making machine learning models easier to understand, received some attention.
- Google's DeepMind created AlphaGo Zero, which accomplished godlike Go abilities to play without human information, utilizing just support learning.

2017:

- Move learning acquired noticeable quality, permitting pretrained models to be utilized for different errands with restricted information.
- Better synthesis and generation of complex data were made possible by the introduction of generative models like variational autoencoders (VAEs) and Wasserstein GANs.
- These are only a portion of the eminent headways and achievements in AI during the predefined period. The field kept on advancing quickly past 2017, with new leap forwards, strategies, and applications arising.

Machine Learning at present:

The field of machine learning has made significant strides in recent years, and its applications are numerous, including self-driving cars, Amazon Alexa, Catboats, and the recommender system. It incorporates clustering, classification, decision tree, SVM algorithms, and reinforcement learning, as well as unsupervised and supervised learning.

Present day AI models can be utilized for making different expectations, including climate expectation, sickness forecast, financial exchange examination, and so on.

Basic Concepts in Machine Learning

Machine Learning is continuously growing in the IT world and gaining strength in different business sectors. Although Machine Learning is in the developing phase, it is popular among all technologies. It is a field of study that makes computers capable of automatically learning and improving from experience. Hence, Machine Learning focuses on the strength of computer programs with the help of collecting data from various observations. In this article, "Concepts in Machine Learning", we will discuss a few basic concepts used in Machine Learning such as what is Machine Learning, technologies and algorithms used in Machine Learning, Applications and example of Machine Learning, and much more. So, let's start with a quick introduction to machine learning.

- **Task:** A task is defined as the main problem in which we are interested. This task/problem can be related to the predictions and recommendations and estimations, etc.
- **Experience:** It is defined as learning from historical or past data and used to estimate and resolve future tasks.
- **Performance:** It is defined as the capacity of any machine to resolve any machine learning task or problem and provide the best outcome for the same. However, performance is dependent on the type of machine learning problems.

Techniques in Machine Learning

Machine Learning techniques are divided mainly into the following 4 categories:

1. Supervised Learning

Supervised learning is applicable when a machine has sample data, i.e., input as well as output data with correct labels. Correct labels are used to check the correctness of the model using some labels and tags. Supervised learning technique helps us to predict future events with the help of past experience and labeled examples. Initially, it analyses the known training dataset, and later it introduces an inferred function that makes predictions about output values. Further, it also predicts errors during this entire learning process and also corrects those errors through algorithms.

Example: Let's assume we have a set of images tagged as "dog". A machine learning algorithm is trained with these dog images so it can easily distinguish whether an image is a dog or not.

2. Unsupervised Learning

In unsupervised learning, a machine is trained with some input samples or labels only, while output is not known. The training information is neither classified nor labeled; hence, a machine may not always provide correct output compared to supervised learning.

Although Unsupervised learning is less common in practical business settings, it helps in exploring the data and can draw inferences from datasets to describe hidden structures from unlabeled data.

Example: Let's assume a machine is trained with some set of documents having different categories (Type A, B, and C), and we have to organize them into appropriate groups. Because the machine is provided only with input samples or without output, so, it can organize these datasets into type A, type B, and type C categories, but it is not necessary whether it is organized correctly or not.

3. Reinforcement Learning

Reinforcement Learning is a feedback-based machine learning technique. In such type of learning, agents (computer programs) need to explore the environment, perform actions, and on the basis of their actions, they get rewards as feedback. For each good action, they get a positive reward, and for each bad action, they get a negative reward. The goal of a Reinforcement learning agent is to maximize the positive rewards. Since there is no labeled data, the agent is bound to learn by its experience only.

4. Semi-supervised Learning

Semi-supervised Learning is an intermediate technique of both supervised and unsupervised learning. It performs actions on datasets having few labels as well as unlabeled data. However, it generally contains unlabeled data. Hence, it also reduces the cost of the machine learning model as labels are costly, but for corporate purposes, it may have few labels. Further, it also increases the accuracy and performance of the machine learning model.

Sem-supervised learning helps data scientists to overcome the drawback of supervised and unsupervised learning. Speech analysis, web content classification, protein sequence classification, text documents classifiers, etc., are some important applications of Semi-supervised learning.

Applications of Machine Learning

Machine Learning is widely being used in approximately every sector, including healthcare, marketing, finance, infrastructure, automation, etc. There are some important real-world examples of machine learning, which are as follows:

Healthcare and Medical Diagnosis:

Machine Learning is used in healthcare industries that help in generating neural networks. These self-learning neural networks help specialists for providing quality treatment by analyzing external data on a patient's condition, X-rays, CT scans, various tests, and screenings. Other than treatment, machine learning is also helpful for cases like automatic billing, clinical decision supports, and development of clinical care guidelines, etc.

Marketing:

Machine learning helps marketers to create various hypotheses, testing, evaluation, and analyze datasets. It helps us to quickly make predictions based on the concept of big data. It is also helpful for stock marketing as most of the trading is done through bots and based on calculations from machine learning algorithms. Various Deep Learning Neural network helps to build trading models such as Convolutional Neural Network, Recurrent Neural Network, Long-short term memory, etc.

Self-driving cars:

This is one of the most exciting applications of machine learning in today's world. It plays a vital role in developing self-driving cars. Various automobile companies like Tesla, Tata, etc., are continuously working for the development of self-driving cars. It also becomes possible by the machine learning method (supervised learning), in which a machine is trained to detect people and objects while driving.

Speech Recognition:

Speech Recognition is one of the most popular applications of machine learning. Nowadays, almost every mobile application comes with a voice search facility. This "Search By Voice" facility is also a part of speech recognition. In this method, voice instructions are converted into text, which is known as Speech to text" or "Computer speech recognition.

Google assistant, SIRI, Alexa, Cortana, etc., are some famous applications of speech recognition.

Traffic Prediction:

Machine Learning also helps us to find the shortest route to reach our destination by using Google Maps. It also helps us in predicting traffic conditions, whether it is cleared or congested, through the real-time location of the Google Maps app and sensor.

Image Recognition:

Image recognition is also an important application of machine learning for identifying objects, persons, places, etc. Face detection and auto friend tagging suggestion is the most famous application of image recognition used by Facebook, Instagram, etc. Whenever we upload photos with our Facebook friends, it automatically suggests their names through image recognition technology.

Product Recommendations:

Machine Learning is widely used in business industries for the marketing of various products. Almost all big and small companies like Amazon, Alibaba, Walmart, Netflix, etc., are using machine learning techniques for products recommendation to their users. Whenever we search for any products on their websites, we automatically get started with lots of advertisements for similar products. This is also possible by Machine Learning algorithms that learn users' interests and, based on past data, suggest products to the user.

Automatic Translation:

Automatic language translation is also one of the most significant applications of machine learning that is based on sequence algorithms by translating text of one language into other desirable languages. Google GNMT (Google Neural Machine Translation) provides this feature, which is Neural Machine Learning. Further, you can also translate the selected text on images as well as complete documents through Google Lens.

Virtual Assistant:

A virtual personal assistant is also one of the most popular applications of machine learning. First, it records out voice and sends to cloud-based server then decode it with the help of machine learning algorithms. All big companies like Amazon, Google, etc., are

using these features for playing music, calling someone, opening an app and searching data on the internet, etc.

Email Spam and Malware Filtering:

Machine Learning also helps us to filter various Emails received on our mailbox according to their category, such as important, normal, and spam. It is possible by ML algorithms such as Multi-Layer Perceptron, Decision tree, and Naïve Bayes classifier.

Commonly used Machine Learning Algorithms

Here is a list of a few commonly used Machine Learning Algorithms as follows:

Linear Regression

Linear Regression is one of the simplest and popular machine learning algorithms recommended by a data scientist. It is used for predictive analysis by making predictions for real variables such as experience, salary, cost, etc.

It is a statistical approach that represents the linear relationship between two or more variables, either dependent or independent, hence called Linear Regression. It shows the value of the dependent variable changes with respect to the independent variable, and the slope of this graph is called as Line of Regression.

Linear Regression can be expressed mathematically as follows:

$$y = a_0 + a_1x + \varepsilon$$

Y= Dependent Variable

X= Independent Variable

a_0 = intercept of the line (Gives an additional degree of freedom)

a_1 = Linear regression coefficient (scale factor to each input value).

ε = random error

The values for x and y variables are training datasets for Linear Regression model representation.

Types of Linear Regression:

- Simple Linear Regression
- Multiple Linear Regression

Applications of Linear Regression:

Linear Regression is helpful for evaluating the business trends and forecasts such as prediction of salary of a person based on their experience, prediction of crop production based on the amount of rainfall, etc.

Logistic Regression

Logistic Regression is a subset of the Supervised learning technique. It helps us to predict the output of categorical dependent variables using a given set of independent variables. However, it can be Binary (0 or 1) as well as Boolean (true/false), but instead of giving an exact value, it gives a probabilistic value between 0 or 1. It is much similar to Linear Regression, depending on its use in the machine learning model. As Linear regression is used for solving regression problems, similarly, Logistic regression is helpful for solving classification problems.

Logistic Regression can be expressed as an 'S-shaped curve called sigmoid functions. It predicts two maximum values (0 or 1).

Mathematically, we can express Logistic regression as follows:

Types of Logistic Regression:

- Binomial
- Multinomial
- Ordinal

K Nearest Neighbour (KNN)

It is also one of the simplest machine learning algorithms that come under supervised learning techniques. It is helpful for solving regression as well as classification problems. It assumes the similarity between the new data and available data and puts the new data

into the category that is most similar to the available categories. It is also known as Lazy Learner Algorithms because it does not learn from the training set immediately; instead, it stores the dataset, and at the time of classification, it performs an action on the dataset. Let's suppose we have a few sets of images of cats and dogs and want to identify whether a new image is of a cat or dog. Then KNN algorithm is the best way to identify the cat from available data sets because it works on similarity measures. Hence, the KNN model will compare the new image with available images and put the output in the cat's category.

Let's understand the KNN algorithm with the below screenshot, where we have to assign a new data point based on the similarity with available data points.

Applications of KNN algorithm in Machine Learning

Including Machine Learning, KNN algorithms are used in so many fields as follows:

- Healthcare and Medical diagnosis
- Credit score checking
- Text Editing
- Hotel Booking
- Gaming
- Natural Language Processing, etc.

K-Means Clustering

K-Means Clustering is a subset of unsupervised learning techniques. It helps us to solve clustering problems by means of grouping the unlabeled datasets into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if $K=2$, there will be two clusters, and for $K=3$, there will be three clusters, and so on.

Decision Tree

Decision Tree is also another type of Machine Learning technique that comes under Supervised Learning. Similar to KNN, the decision tree also helps us to solve classification as well as regression problems, but it is mostly preferred to solve classification problems. The name decision tree is because it consists of a tree-structured classifier in which

attributes are represented by internal nodes, decision rules are represented by branches, and the outcome of the model is represented by each leaf of a tree. The tree starts from the decision node, also known as the root node, and ends with the leaf node.

Decision nodes help us to make any decision, whereas leaves are used to determine the output of those decisions.

A Decision Tree is a graphical representation for getting all the possible outcomes to a problem or decision depending on certain given conditions.

Random Forest

Random Forest is also one of the most preferred machine learning algorithms that come under the Supervised Learning technique. Similar to KNN and Decision Tree, It also allows us to solve classification as well as regression problems, but it is preferred whenever we have a requirement to solve a complex problem and to improve the performance of the model.

A random forest algorithm is based on the concept of ensemble learning, which is a process of combining multiple classifiers.

Random forest classifier is made from a combination of a number of decision trees as well as various subsets of the given dataset. This combination takes input as an average prediction from all trees and improves the accuracy of the model. The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting. Further, It also takes less training time as compared to other algorithms.

Support Vector Machines (SVM)

It is also one of the most popular machine learning algorithms that come as a subset of the Supervised Learning technique in machine learning. The goal of the support vector machine algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane. It is also used to solve classification as well as regression problems. It is used for Face detection, image classification, text categorization, etc.

Naïve Bayes

The naïve Bayes algorithm is one of the simplest and most effective machine learning algorithms that come under the supervised learning technique. It is based on the concept of the Bayes Theorem, used to solve classification-related problems. It helps to build fast machine learning models that can make quick predictions with greater accuracy and performance. It is mostly preferred for text classification having high-dimensional training datasets.

It is used as a probabilistic classifier which means it predicts on the basis of the probability of an object. Spam filtration, Sentimental analysis, and classifying articles are some important applications of the Naïve Bayes algorithm.

It is also based on the concept of Bayes Theorem, which is also known as Bayes' Rule or Bayes' law. Mathematically, Bayes Theorem can be expressed as follows:

Where,

- $P(A)$ is Prior Probability
- $P(B)$ is Marginal Probability
- $P(A|B)$ is Posterior probability
- $P(B|A)$ is Likelihood probability

Difference between machine learning and Artificial Intelligence

- Artificial intelligence is a technology using which we can create intelligent systems that can simulate human intelligence, whereas Machine learning is a subfield of artificial intelligence, which enables machines to learn from past data or experiences.
- Artificial Intelligence is a technology used to create an intelligent system that enables a machine to simulate human behavior. Whereas, Machine Learning is a branch of AI which helps a machine to learn from experience without being explicitly programmed.
- AI helps to make humans like intelligent computer systems to solve complex problems. Whereas, ML is used to gain accurate predictions from past data or experience.
- AI can be divided into Weak AI, General AI, and Strong AI. Whereas, IML can be divided into Supervised learning, Unsupervised learning, and Reinforcement learning.

- Each AI agent includes learning, reasoning, and self-correction. Each ML model includes learning and self-correction when introduced with new data.
- AI deals with Structured, semi-structured, and unstructured data. ML deals with Structured and semi-structured data.
- **Applications of AI:** Siri, customer support using chatbots, Expert System, Online game playing, an intelligent humanoid robot, etc. **Applications of ML:** Online recommender system, Google search algorithms, Facebook auto friend tagging suggestions, etc.

Aspects of developing a learning system: training data, concept representation, function approximation

According to Arthur Samuel "Machine Learning enables a Machine to Automatically learn from Data, Improve performance from an Experience and predict things without explicitly programmed."

In Simple Words, When we feed the Training Data to Machine Learning Algorithm, this algorithm will produce a mathematical model and with the help of the mathematical model, the machine will make a prediction and take a decision without being explicitly programmed. Also, during training data, the more machine will work with it the more it will get experience and the more efficient result is produced.

The following are the different aspects of developing a learning system. Let us consider designing a program to learn to play checkers, with the goal of entering it in the world checkers tournament.

Example : In Driverless Car, the training data is fed to Algorithm like how to Drive Car in Highway, Busy and Narrow Street with factors like speed limit, parking, stop at signal etc. After that, a Logical and Mathematical model is created on the basis of that and after that, the car will work according to the logical model. Also, the more data the data is fed the more efficient output is produced.

Designing a Learning System in Machine Learning :

According to Tom Mitchell, "A computer program is said to be learning from experience (E), with respect to some task (T). Thus, the performance measure (P) is the performance at task T, which is measured by P, and it improves with experience E."

Example: In Spam E-Mail detection,

- **Task, T:** To classify mails into Spam or Not Spam.
- **Performance measure, P:** Total percent of mails being correctly classified as being "Spam" or "Not Spam".
- **Experience, E:** Set of Mails with label "Spam"

Checkers learning problem

- Task T: playing checkers
- Performance measure P: percent of games won against opponents
- Training experience E: playing practice games against itself

In order to complete the design of the learning system, we must now choose

- The exact type of knowledge to be learn
- A representation for this target knowledge
- A learning mechanism

1. Choosing the training experience

- The first design choice is to choose the type of training experience from which the system will learn.
- The type of training experience available can have a significant impact on success or failure of the learner.

There are three attributes which impact on success or failure of the learner

1. Whether the training experience provides direct or indirect feedback regarding the choices made by the performance system
2. The degree to which the learner controls the sequence of training examples
3. How well it represents the distribution of examples over which the final system performance P must be measured

2. Choosing the Target Function

The next important step is choosing the target function. It means according to the knowledge fed to the algorithm the machine learning will choose NextMove function which will describe what type of legal moves should be taken. For example : While playing chess with the opponent, when opponent will play then the machine learning algorithm will decide what be the number of possible legal moves taken in order to get success.

NextMove: $B \rightarrow M$

This function accepts as input any board from the set of legal board states B and produces as output some move from the set of legal moves M.

An alternative target function and one that will turn out to be easier to learn in this setting is an evaluation function that assigns a numerical score to any given board state.

V: $B \rightarrow R$

Denote that V maps any legal board state from the set B to some real value

Let us therefore define the target value $V(b)$ for an arbitrary board state b in B , as follows

1. if b is a final board state that is won, then $V(b) = 100$
2. if b is a final board state that is lost, then $V(b) = -100$
3. if b is a final board state that is drawn, then $V(b) = 0$
4. if b is not a final state in the game, then $V(b) = V(b')$, where b' is the best final board state that can be achieved starting from b and playing optimally until the end of the game.

3. Choosing a representation for the target function

We need to choose a representation that the learning algorithm will use to describe the function NextMove. The function NextMove will be calculated as a linear combination of the following board features:

- x_1 : the number of black pieces on the board
- x_2 : the number of red pieces on the board
- x_3 : the number of black kings on the board
- x_4 : the number of red kings on the board
- x_5 : the number of black pieces threatened by red (i.e., which can be captured on red's next turn)
- x_6 : the number of red pieces threatened by black

$$\text{NextMove} = u_0 + u_1x_1 + u_2x_2 + u_3x_3 + u_4x_4 + u_5x_5 + u_6x_6$$

Here u_0, u_1 up to u_6 are the coefficients that will be chosen(learned) by the learning algorithm

4. Choose a Function Approximation Algorithm

To learn the target function NextMove, we require a set of training examples, each describing a specific board state b and the training value (Correct Move) y for b . The training algorithm learns/approximates the coefficients u_0, u_1 up to u_6 with the help of these training examples by estimating and adjusting these weights.

For Example: When a training data of Playing chess is fed to algorithm so at that time it is not machine algorithm will fail or get success and again from that failure or success it will measure while next move what step should be chosen and what is its success rate.

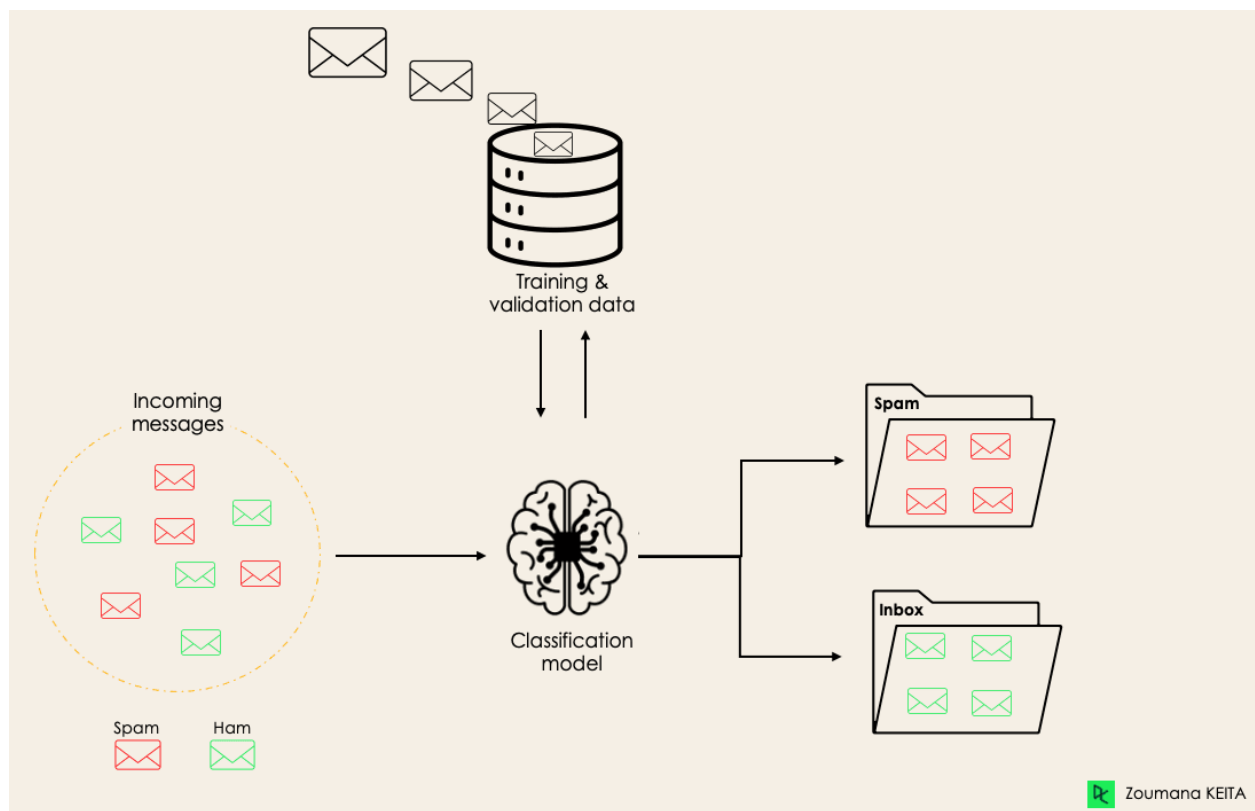
5. The final design

The final design is created at last when system goes from number of examples, failures and success, correct and incorrect decision and what will be the next step etc.

Overview Of classification

Classification is a supervised machine learning method where the model tries to predict the correct label of a given input data. In classification, the model is fully trained using the training data, and then it is evaluated on test data before being used to perform prediction on new unseen data.

For instance, an algorithm can learn to predict whether a given email is spam or ham (no spam), as illustrated below.



Before diving into the classification concept, we will first understand the difference between the two types of learners in classification: lazy and eager learners. Then we will clarify the misconception between classification and regression.

Lazy Learners Vs. Eager Learners

There are two types of learners in machine learning classification: lazy and eager learners.

Eager learners are machine learning algorithms that first build a model from the training dataset before making any prediction on future datasets. They spend more time during the training process because of their eagerness to have a better generalization during the training from learning the weights, but they require less time to make predictions.

Most machine learning algorithms are eager learners, and below are some examples:

- Logistic Regression.
- Support Vector Machine.
- Decision Trees.
- Artificial Neural Networks.

Lazy learners or instance-based learners, on the other hand, do not create any model immediately from the training data, and this is where the lazy aspect comes from. They just memorize the training data, and each time there is a need to make a prediction, they search for the nearest neighbor from the whole training data, which makes them very slow during prediction. Some examples of this kind are:

- K-Nearest Neighbor.
- Case-based reasoning.

However, some algorithms, such as [BallTrees](#) and [KDTrees](#), can be used to improve the prediction latency.

Machine Learning Classification Vs. Regression

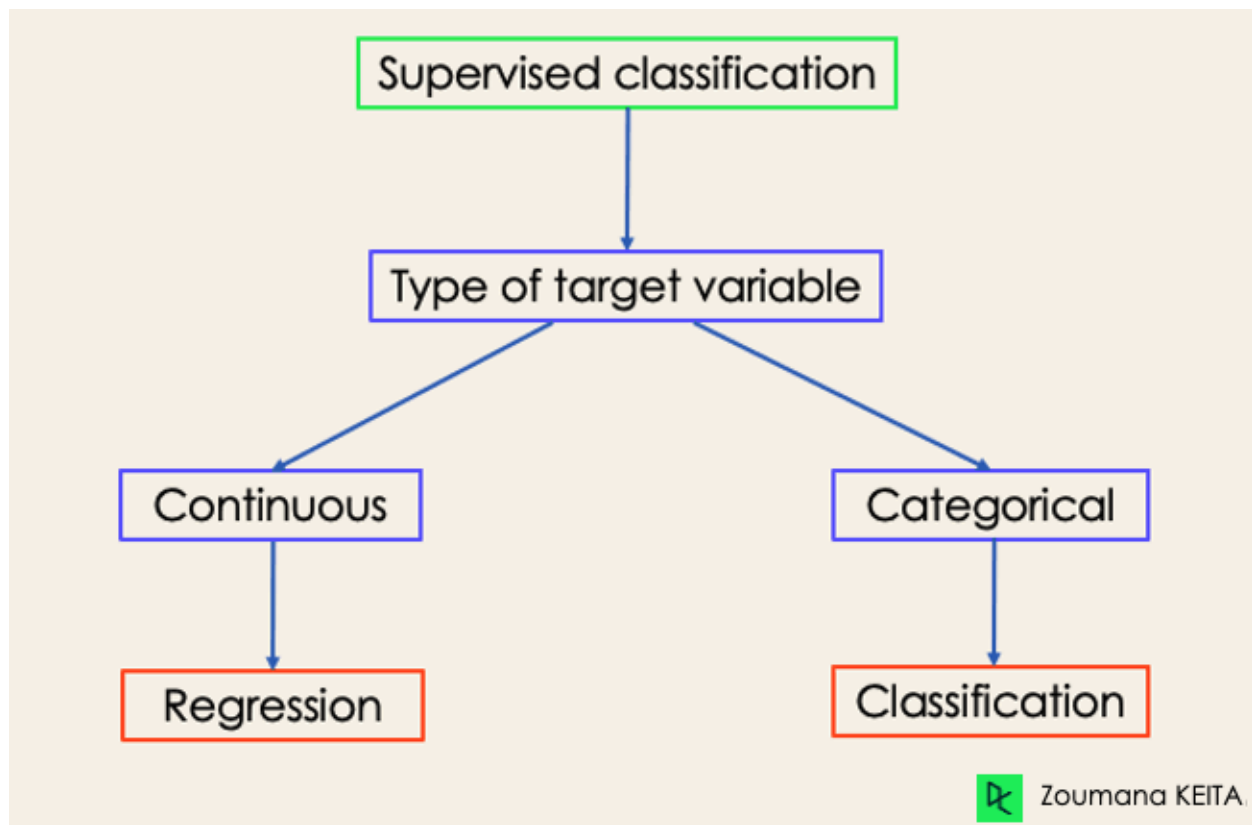
There are four main categories of Machine Learning algorithms: supervised, unsupervised, semi-supervised, and reinforcement learning.

Even though classification and regression are both from the category of supervised learning, they are not the same.

- The prediction task is a **classification** when the target variable is discrete. An application is the identification of the underlying sentiment of a piece of text.
- The prediction task is a **regression** when the target variable is continuous. An example can be the prediction of the salary of a person given their education degree, previous work experience, geographical location, and level of seniority.

If you are interested in knowing more about classification, courses on [Supervised Learning with scikit-learn](#) and [Supervised Learning in R might be helpful](#). They provide you with a better understanding of how each algorithm approaches tasks and the Python and R functions required to implement them.

Regarding regression, [Introduction to Regression in R](#) and [Introduction to Regression with statsmodels in Python](#) will help you explore different types of regression models as well as their implementation in R and Python.



Examples of Machine Learning Classification in Real Life

Supervised Machine Learning Classification has different applications in multiple domains of our day-to-day life. Below are some examples.

Healthcare

Training a machine learning model on historical patient data can help healthcare specialists accurately analyze their diagnoses:

- During the COVID-19 pandemic, machine learning models were implemented to efficiently predict whether a person had COVID-19 or not.
- Researchers can use machine learning models to predict new diseases that are more likely to emerge in the future.

Education

Education is one of the domains dealing with the most textual, video, and audio data. This unstructured information can be analyzed with the help of Natural Language technologies to perform different tasks such as:

- The classification of documents per category.
- Automatic identification of the underlying language of students' documents during their application.
- Analysis of students' feedback sentiments about a Professor.

Transportation

Transportation is the key component of many countries' economic development. As a result, industries are using machine and deep learning models:

- To predict which geographical location will have a rise in traffic volume.
- Predict potential issues that may occur in specific locations due to weather conditions.

Sustainable agriculture

Agriculture is one of the most valuable pillars of human survival. Introducing sustainability can help improve farmers' productivity at a different level without damaging the environment:

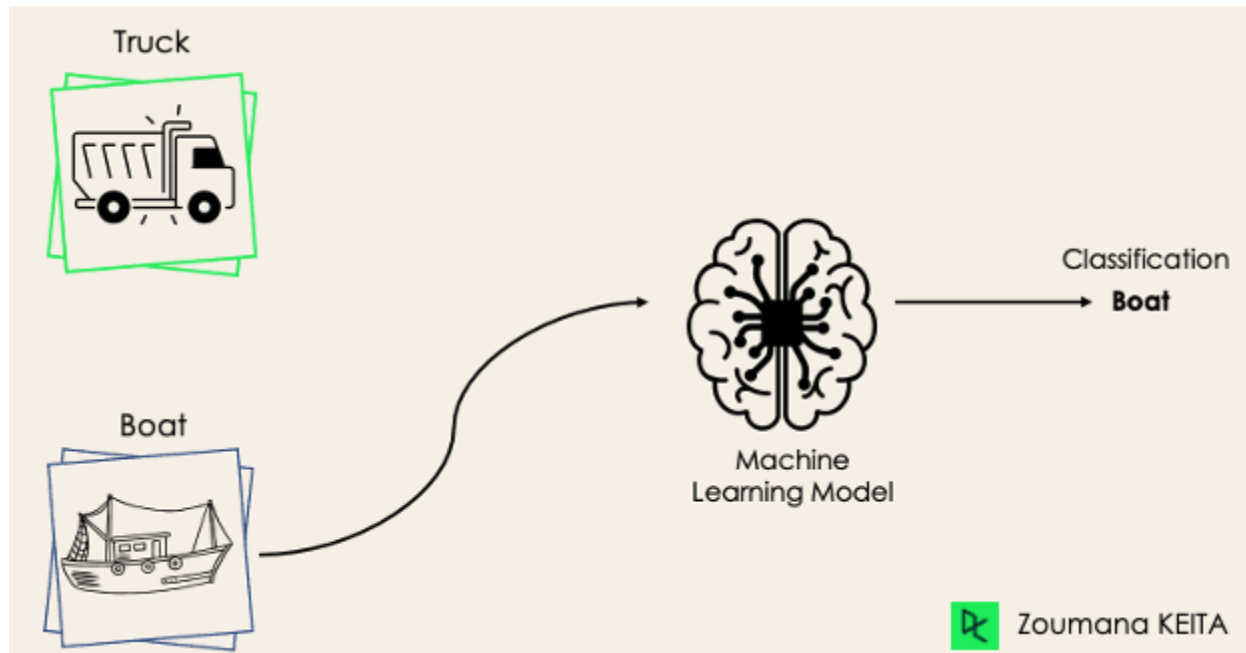
- By using classification models to predict which type of land is suitable for a given type of seed.
- Predict the weather to help them take proper preventive measures.

Different Types of Classification Tasks in Machine Learning

There are four main classification tasks in Machine learning: binary, multi-class, multi-label, and imbalanced classifications.

Binary Classification

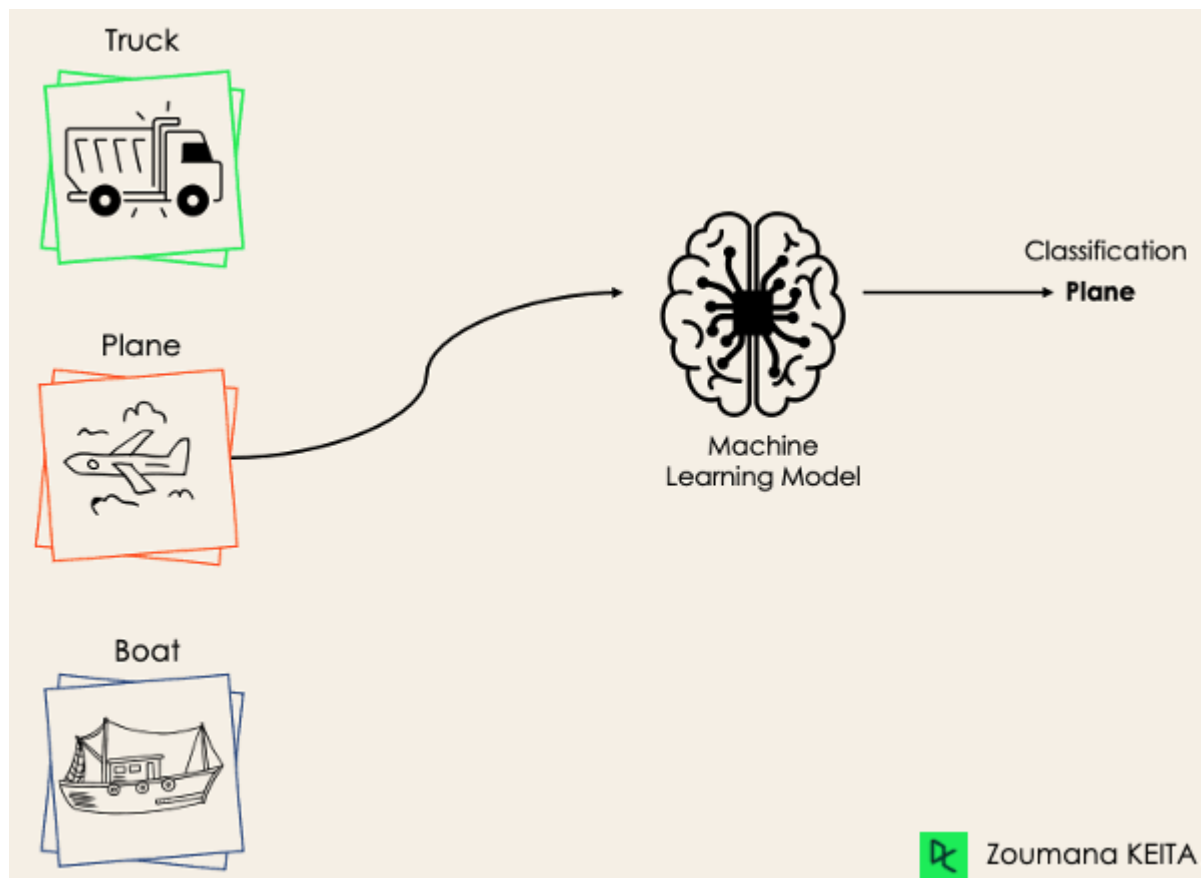
In a binary classification task, the goal is to classify the input data into two mutually exclusive categories. The training data in such a situation is labeled in a binary format: true and false; positive and negative; 0 and 1; spam and not spam, etc. depending on the problem being tackled. For instance, we might want to detect whether a given image is a truck or a boat.



Logistic Regression and Support Vector Machines algorithms are natively designed for binary classifications. However, other algorithms such as K-Nearest Neighbors and Decision Trees can also be used for binary classification.

Multi-Class Classification

The multi-class classification, on the other hand, has at least two mutually exclusive class labels, where the goal is to predict to which class a given input example belongs to. In the following case, the model correctly classified the image to be a plane.



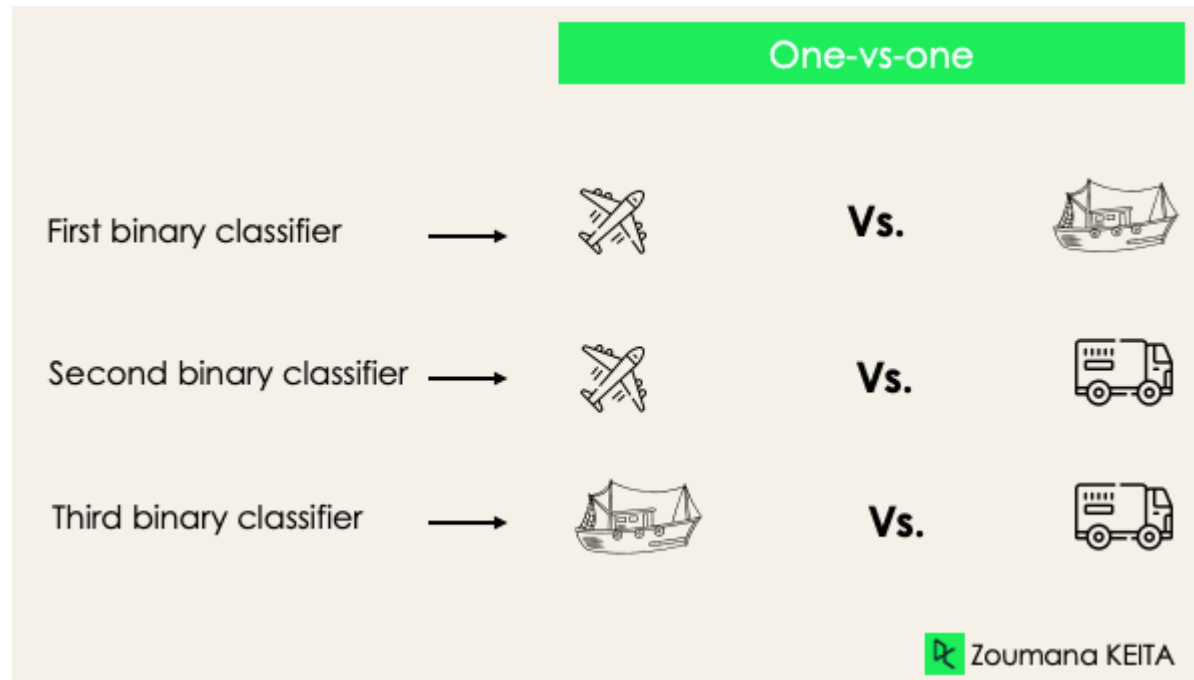
Most of the binary classification algorithms can be also used for multi-class classification. These algorithms include but are not limited to:

- Random Forest
- Naive Bayes
- K-Nearest Neighbors
- Gradient Boosting
- SVM
- Logistic Regression.

But wait! Didn't you say that SVM and Logistic Regression do not support multi-class classification by default?

→ That's correct. However, we can apply binary transformation approaches such as one-versus-one and one-versus-all to adapt native binary classification algorithms for multi-class classification tasks.

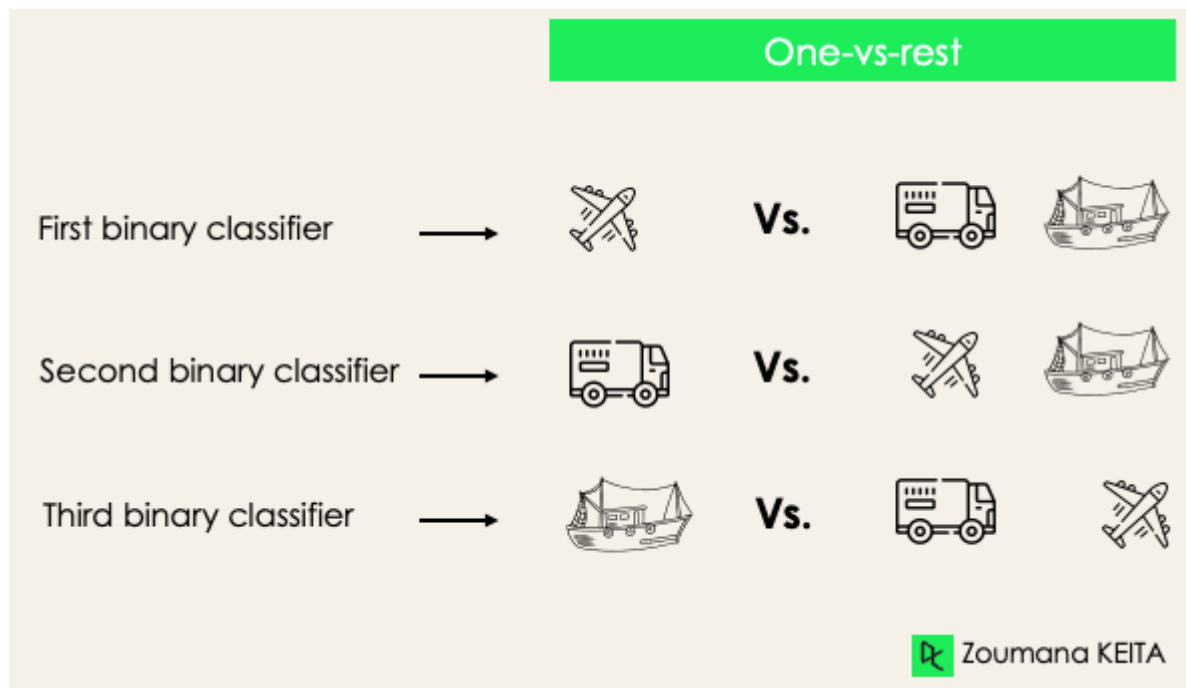
One-versus-one: this strategy trains as many classifiers as there are pairs of labels. If we have a 3-class classification, we will have three pairs of labels, thus three classifiers, as shown below.



In general, for N labels, we will have $N \times (N-1)/2$ classifiers. Each classifier is trained on a single binary dataset, and the final class is predicted by a majority vote between all the classifiers. One-vs-one approach works best for SVM and other kernel-based algorithms.

One-versus-rest: at this stage, we start by considering each label as an independent label and consider the rest combined as only one label. With 3-classes, we will have three classifiers.

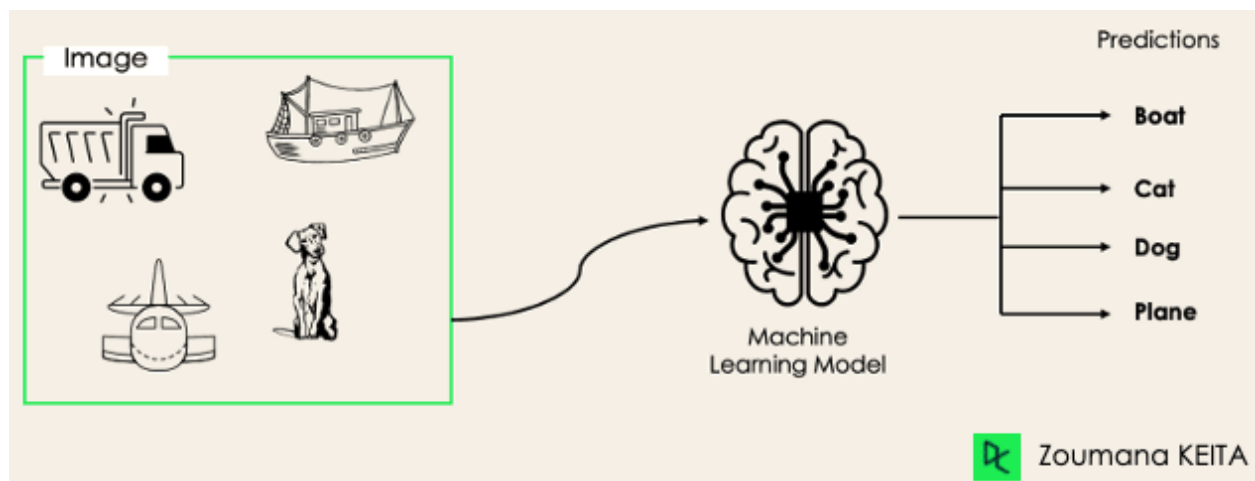
In general, for N labels, we will have N binary classifiers.



Multi-Label Classification

In multi-label classification tasks, we try to predict 0 or more classes for each input example. In this case, there is no mutual exclusion because the input example can have more than one label.

Such a scenario can be observed in different domains, such as auto-tagging in Natural Language Processing, where a given text can contain multiple topics. Similarly to computer vision, an image can contain multiple objects, as illustrated below: the model predicted that the image contains: a plane, a boat, a truck, and a dog.

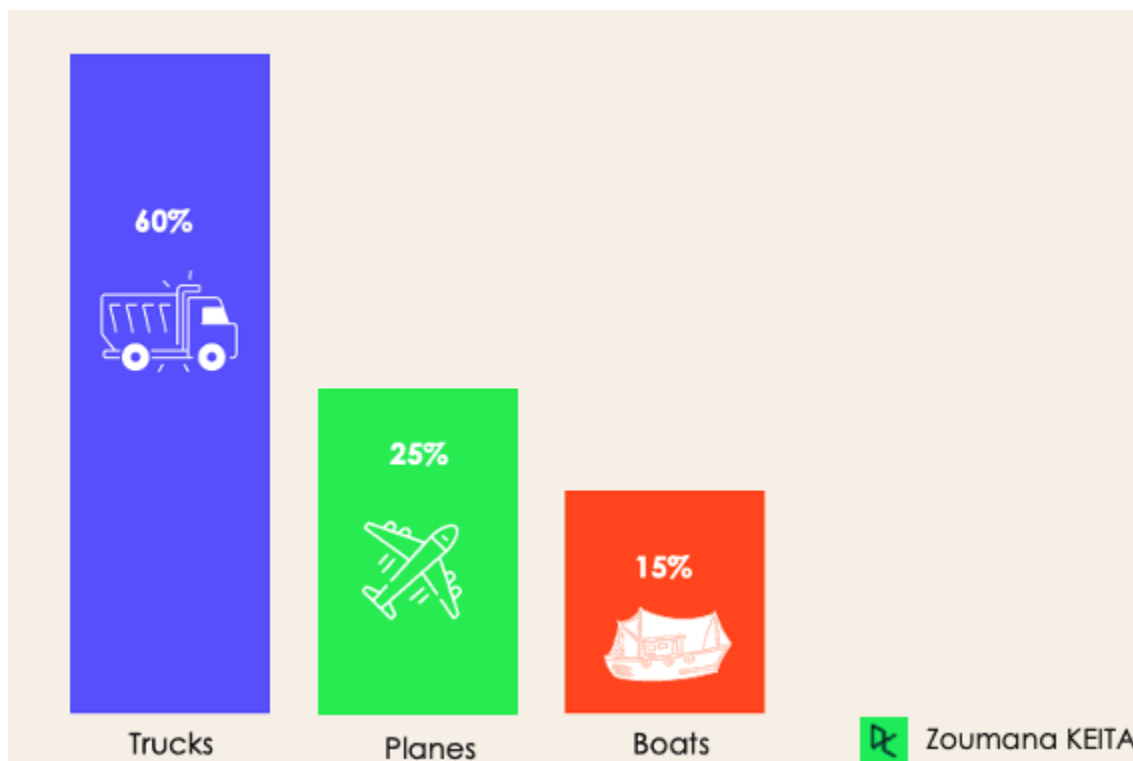


It is not possible to use multi-class or binary classification models to perform multi-label classification. However, most algorithms used for those standard classification tasks have their specialized versions for multi-label classification. We can cite:

- *Multi-label Decision Trees*
- *Multi-label Gradient Boosting*
- *Multi-label Random Forests*

Imbalanced Classification

For the imbalanced classification, the number of examples is unevenly distributed in each class, meaning that we can have more of one class than the others in the training data. Let's consider the following 3-class classification scenario where the training data contains: 60% of trucks, 25% of planes, and 15% of boats.



The imbalanced classification problem could occur in the following scenario:

- *Fraudulent transaction detections in financial industries*
- *Rare disease diagnosis*
- *Customer churn analysis*

Using conventional predictive models such as Decision Trees, Logistic Regression, etc. could not be effective when dealing with an imbalanced dataset, because they might be biased toward predicting the class with the highest number of observations, and considering those with fewer numbers as noise.

So, does that mean that such problems are left behind?

Of course not! We can use multiple approaches to tackle the imbalance problem in a dataset. The most commonly used approaches include sampling techniques or harnessing the power of cost-sensitive algorithms.

Sampling Techniques

These techniques aim to balance the distribution of the original by:

- *Cluster-based Oversampling:*
- *Random undersampling: random elimination of examples from the majority class.*
- *SMOTE Oversampling: random replication of examples from the minority class.*

Cost-Sensitive Algorithms

These algorithms take into consideration the cost of misclassification. They aim to minimize the total cost generated by the models.

- *Cost-sensitive Decision Trees.*
- *Cost-sensitive Logistic Regression.*
- *Cost-sensitive Support Vector Machines.*

Metrics to Evaluate Machine Learning Classification Algorithms

Now that we have an idea of the different types of classification models, it is crucial to choose the right evaluation metrics for those models. In this section, we will cover the most commonly used metrics: accuracy, precision, recall, F1 score, and area under the ROC (Receiver Operating Characteristic) curve and AUC (Area Under the Curve).

A bit of context

Imagine you are a healthcare startup, and want an AI assistant able to predict whether a given patient has a heart disease or not based on its health record. This is a binary classification problem where the model will predict

- 1, True or Yes if the patient has heart disease
- 0, False or No otherwise

1 Confusion matrix

A 2X2 matrix that nicely summarizes the number of correct predictions of the model. It also helps in computing different other performance metrics.

Predicti Reality	Yes	No
Yes	True Positives (TP)	False Negatives (FN)
No	False Positives (FP)	True Negatives (TN)

Type I Error

Type II Error

Type I & II Errors can be used interchangeably when referring to False Positives and False negatives respectively

2 Accuracy

We get accuracy by answering this question: "out of the predictions made by the model, what percentage is correct?"

$$\text{Accuracy} = \frac{TP + TN}{\text{Total number observation}}$$

3 Precision

We get precision by answering this question: "out of all the YES predictions, how many of them were correct?"

$$\text{Precision} = \frac{TP}{TP + FP}$$

4 Recall / Sensitivity

It aims to answer this question: "how good was the model at predicting real Yes events?", which can be considered as the flip of the precision.

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

5 Recall / Specificity

It aims to answer this question: "how good was the model at predicting real No events?".

$$\text{Specificity} = \frac{TN}{TN + FP}$$

6 F1 Score

Sometimes used when dealing with imbalanced data set, meaning that there are more of one class/label than there are of the other. It corresponds to the harmonic mean of the precision and recall.

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

7 AUC – ROC Curve

AUC– ROC generates probability values instead of binary 0/1 values. It should be used when your data set is roughly balanced.

Using ROC for imbalanced data sets lead to incorrect interpretation.

ROC curves provide good overview of trade-off between the TP rate and FP rate for binary classifier using different probability thresholds.

- A value below 0.5 indicates a poor classifier
- A value of 0.5 means random classifier
- Value over 0.7 corresponds to a good classifier
- 0.8 indicates a strong classifier
- We have 1 when the classifier perfectly predicts everything.

Strategies to choose the right metric

Choose accuracy

- The cost of FP and FN are roughly equal.
- The benefit of TP and TN are roughly equal.

Choose Precision

- The cost of FP is much higher than a FN.
- The benefit of a TP is much higher than a TN.

Choose recall

- The cost of FN is much higher than a FP.
- The cost of a TN is much higher than a TP.

ROC AUC & Precision – Recall curves

- Use ROC when the dealing with balanced data sets.
- Use precision-recall for imbalanced data sets.



In most supervised machine learning tasks, best practice recommends to split your data into three independent sets: a **training set**, a **testing set**, and a **validation set**.

To learn why, let's pretend that we have a dataset of two types of pets:

Cats: Dogs:

Each pet in our dataset has two features: **weight** and **fluffiness**.

Our goal is to identify and evaluate suitable models for classifying a given pet as either a cat or a dog. We'll use train/test/validations splits to do this!

Train, Test, and Validation Splits

The first step in our classification task is to randomly split our pets into three independent sets:

Training Set: The dataset that we feed our model to learn potential underlying patterns and relationships.

Validation Set: The dataset that we use to understand our model's performance across different model types and hyper parameter choices.

Test Set: The dataset that we use to approximate our model's unbiased accuracy in the wild.

The Training Set

The training set is the dataset that we employ to train our model. It is this dataset that our model uses to learn any underlying patterns or relationships that will enable making predictions later on.

The training set should be as representative as possible of the population that we are trying to model. Additionally, we need to be careful and ensure that it is as unbiased as possible, as any bias at this stage may be propagated downstream during inference.

Building Our Model

Our goal (to determine whether a given pet is a cat or a dog) is a binary classification task, so we will use a simple but effective model appropriate for this task: **logistic regression**.

Logistic regression will learn a decision boundary to best separate the cats from dogs in our training data, using the selected feature (**None**, **Weight**, **Fluffiness**, or *both* **Weight** and **Fluffiness**).

Select the feature to visualize the corresponding logistic regression model's decision boundary. **Drag each animal in the training set to a new position to see how the boundary updates!**

The Validation Set

We can build four different logistic regression models (one for each feature possibility), **how do we decide which model to select?**

We could compare the accuracy of each model on the training set, but if we use the same exact dataset for both training and tuning, the model will overfit and won't generalize well.

This is where the validation set comes in — it acts as an independent, unbiased dataset for comparing the performance of different algorithms trained on our training set.

Select a feature to view the model's performance on the validation set in the table below. **Drag the pets across the line to see how the model performance updates!**

The Testing Set

Once we have used the validation set to determine the algorithm and parameter choices that we would like to use in production, the test set is used to approximate the models's true performance in the wild. It is the final step in evaluating our model's performance on unseen data.

We should never, under any circumstance, look at the test set's performance before selecting a model.

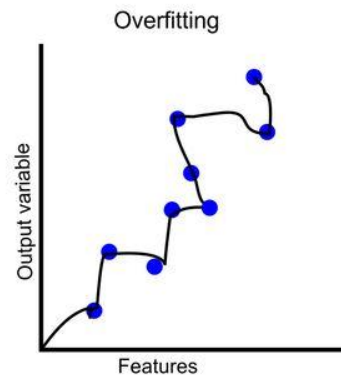
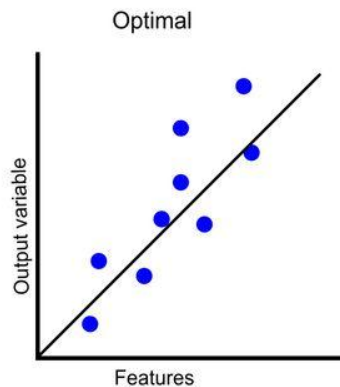
Peeking at our test set performance ahead of time is a form of overfitting, and will likely lead to unreliable performance expectations in production. It should only be checked as the final form of evaluation, after the validation set has been used to identify the best model.

What is Over fitting?

In over fitting, a model becomes so good at our training data that it has mastered every pattern, including noise. This makes the model perform well with training data but poorly with test or validation data.

The illustration below depicts how an optimal model fits into the data compared to over fitting.

In the graph, we have our features on the x-axis. In datasets, features are data that can be used to predict an outcome. The output variable is the outcome based on those features. The blue dots represent the data points where the features determine output variables.



In the optimal graph, our model tries to find the generalized trend. But in our overfitted chart, the model tries to master each data point, resulting in an asymmetrical curve.

An example of a case study would be to predict if a customer would default on a bank loan. Assuming we have a dataset of 100,000 customers containing features such as demographics, income, loan amount, credit history, employment record, and default status, we split our data into training and test data.

Our training dataset contains 80,000 customers, while our test dataset contains 20,000 customers. In the training the dataset, we observe that our model has a 97% accuracy, but in prediction, we only get 50% accuracy. This shows that we have an overfitting problem.

Can you tell why overfitting is a problem? Yes! It produces an incorrect prediction. It is the purpose of machine learning models to make predictions to help business decision-making. We waste time and resources when our model makes incorrect predictions.

Imagine predicting that a customer will pay back a loan, and the customer defaults. Not just one customer but thousands of customers. This can cause a crisis for any financial institution.

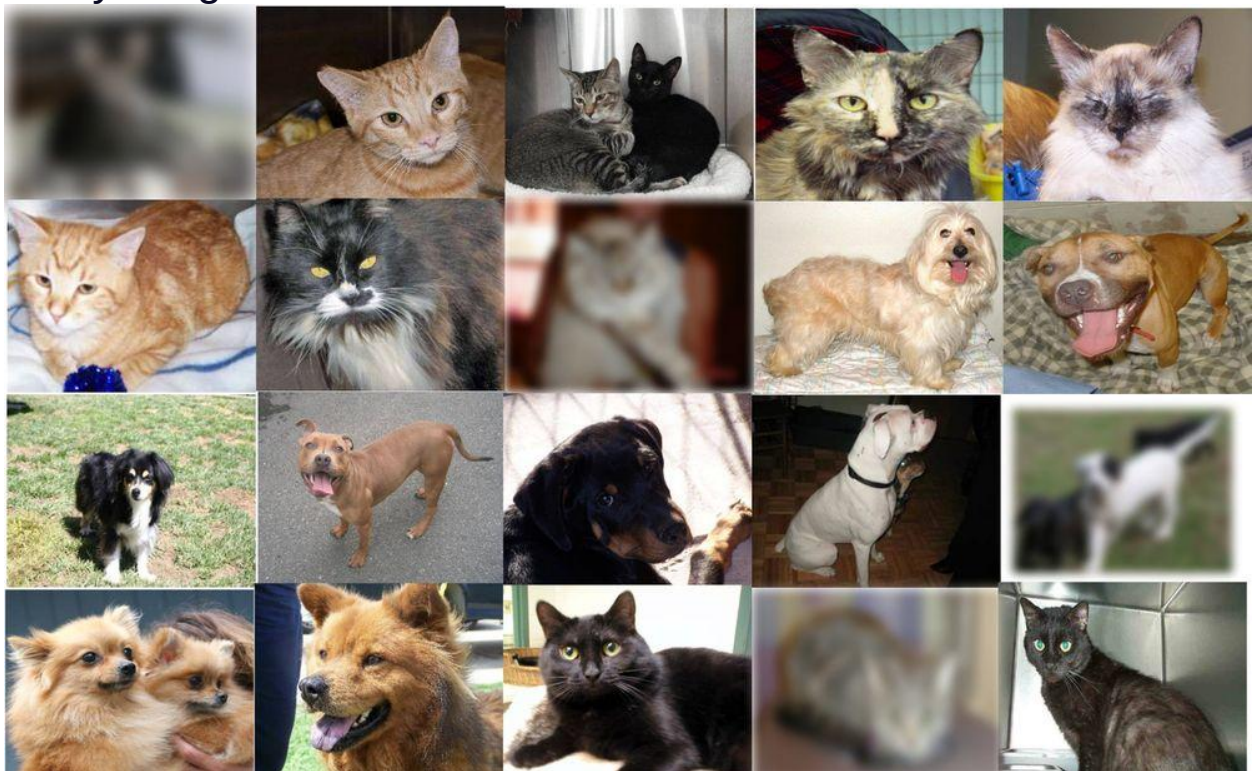
Causes of Overfitting

Noisy data

Noise in data often appears as errors, fluctuations, or outliers in the data. This can be caused by data entry errors, data aging, data transmission errors, and so on.

Too much noise in data can cause the model to think these are valid data points. Fitting the noise pattern in the training dataset will cause poor performance on the new dataset.

For example, let's say that we are building a machine-learning model to classify images of cats and dogs. But some of the images in the dataset are blurry or poorly lit. While the model may perform well on the training data, it might struggle on the test data since it must have mastered some pattern with the blurry images in the dataset.



In the picture above, you can see that we have some blurry images that cannot be labelled if they are cat or dog. In these instances, the model could also learn these patterns alongside relevant features. Removing these images can reduce overfitting.

Insufficient training data

There will be fewer patterns and noises to analyze if we don't have sufficient training data. This means that the machine can only learn a little about our data.

Using our previous example, if our training data contains fewer images of dogs but many more of cats, the model learns so much about cats that when we feed the system an image of a dog, it will likely give a wrong output.

Overly complex model

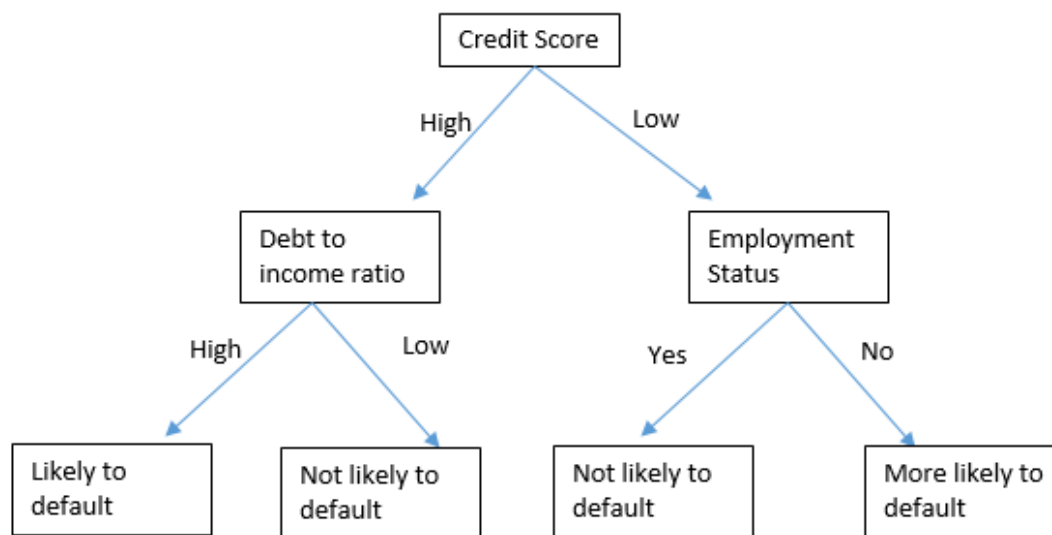
In a complex model, there are many parameters capable of capturing patterns and relationships in training data. As a result, our model makes a more accurate prediction.

But this can pose a problem, since the model can start capturing noise, fluctuations, or outliers. Let's look at a decision tree model, how it works, and how overfitting can happen when it becomes too complex.

A decision tree model works by repeatedly breaking down data into significant features, making each point a node. This creates a tree like structure.

To make a prediction, it starts from the root node and follow the branches down, breaking and fitting every feature until it gets to the leaf node. The prediction is then made based on the value associated with the leaf node.

Let's look at a simple tree diagram of how a decision tree can predict if a customer is likely to default on loan base on certain features.



Tree

diagram showing whether a customer is likely to default on a loan

This model starts by creating a parent node which is credit score. Depending on whether the credit score for the applicant is high or low, it goes down to the next node, which is either debt to income ratio or employment status. Then it makes the final prediction as to whether the customer is likely to default or not.

A decision tree can become overly complex when it creates too many nodes, making it too detailed or specific to the training data.

Linear Discriminant Analysis (LDA) in Machine Learning

Linear Discriminant Analysis (LDA) is one of the commonly used dimensionality reduction techniques in machine learning to solve more than two-class classification problems. It is also known as Normal Discriminant Analysis (NDA) or Discriminant Function Analysis (DFA).

This can be used to project the features of higher dimensional space into lower-dimensional space in order to reduce resources and dimensional costs. In this topic,

"**Linear Discriminant Analysis (LDA) in machine learning**", we will discuss the LDA algorithm for classification predictive modeling problems, limitation of logistic regression, representation of linear Discriminant analysis model, how to make a prediction using LDA, how to prepare data for LDA, extensions to LDA and much more. So, let's start with a quick introduction to Linear Discriminant Analysis (LDA) in machine learning.

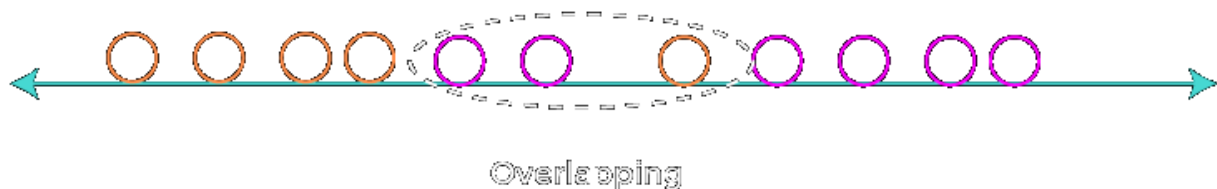
Note: Before starting this topic, it is recommended to learn the basics of Logistic Regression algorithms and a basic understanding of classification problems in machine learning as a prerequisite

What is Linear Discriminant Analysis (LDA)?

Although the logistic regression algorithm is limited to only two-class, linear Discriminant analysis is applicable for more than two classes of classification problems.

Linear Discriminant analysis is one of the most popular dimensionality reduction techniques used for supervised classification problems in machine learning. It is also considered a pre-processing step for modeling differences in ML and applications of pattern classification.

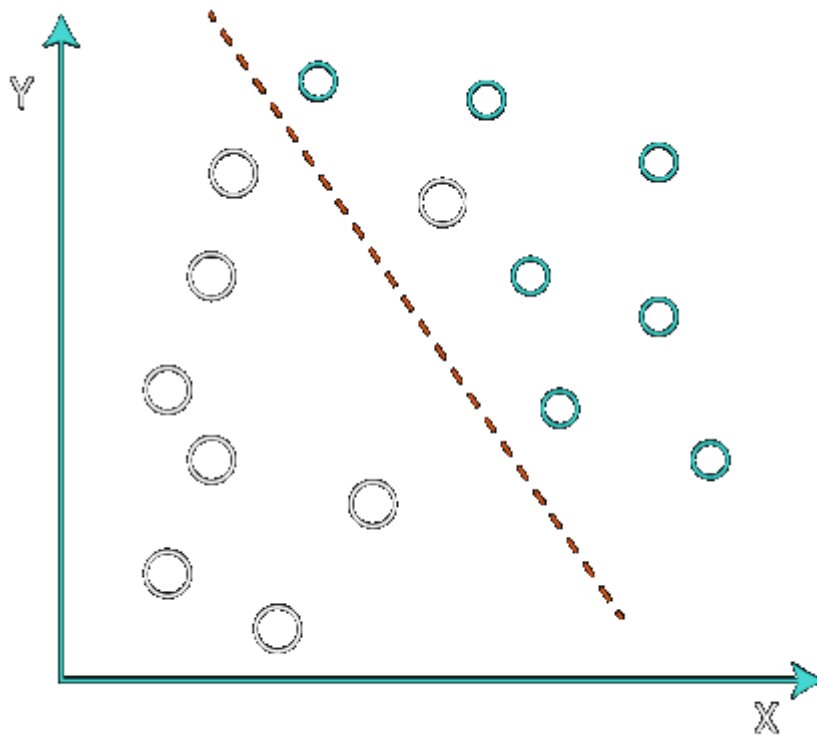
Whenever there is a requirement to separate two or more classes having multiple features efficiently, the Linear Discriminant Analysis model is considered the most common technique to solve such classification problems. For e.g., if we have two classes with multiple features and need to separate them efficiently. When we classify them using a single feature, then it may show overlapping.



To overcome the overlapping issue in the classification process, we must increase the number of features regularly.

Example:

Let's assume we have to classify two different classes having two sets of data points in a 2-dimensional plane as shown below image:



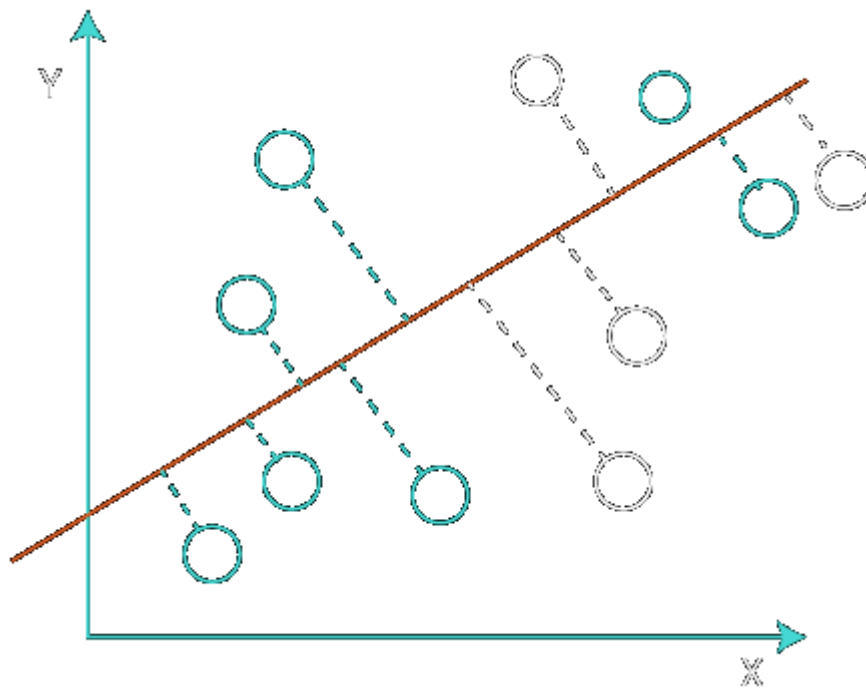
However, it is impossible to draw a straight line in a 2-d plane that can separate these data points efficiently but using linear Discriminant analysis; we can dimensionally reduce the 2-D plane into the 1-D plane. Using this technique, we can also maximize the separability between multiple classes.

How Linear Discriminant Analysis (LDA) works?

Linear Discriminant analysis is used as a dimensionality reduction technique in machine learning, using which we can easily transform a 2-D and 3-D graph into a 1-dimensional plane.

Let's consider an example where we have two classes in a 2-D plane having an X-Y axis, and we need to classify them efficiently. As we have already seen in the above example that LDA enables us to draw a straight line that can completely separate the two classes of the data points. Here, LDA uses an X-Y axis to create a new axis by separating them using a straight line and projecting data onto a new axis.

Hence, we can maximize the separation between these classes and reduce the 2-D plane into 1-D.



To create a new axis, Linear Discriminant Analysis uses the following criteria:

- It maximizes the distance between means of two classes.
- It minimizes the variance within the individual class.

Using the above two conditions, LDA generates a new axis in such a way that it can maximize the distance between the means of the two classes and minimizes the variation within each class.

In other words, we can say that the new axis will increase the separation between the data points of the two classes and plot them onto the new axis.

Why LDA?

- Logistic Regression is one of the most popular classification algorithms that perform well for binary classification but falls short in the case of multiple classification problems with well-separated classes. At the same time, LDA handles these quite efficiently.
- LDA can also be used in data pre-processing to reduce the number of features, just as PCA, which reduces the computing cost significantly.

- LDA is also used in face detection algorithms. In Fisherfaces, LDA is used to extract useful data from different faces. Coupled with eigenfaces, it produces effective results.

Drawbacks of Linear Discriminant Analysis (LDA)

Although, LDA is specifically used to solve supervised classification problems for two or more classes which are not possible using logistic regression in machine learning. But LDA also fails in some cases where the Mean of the distributions is shared. In this case, LDA fails to create a new axis that makes both the classes linearly separable.

To overcome such problems, we use **non-linear Discriminant analysis** in machine learning.

Extension to Linear Discriminant Analysis (LDA)

Linear Discriminant analysis is one of the most simple and effective methods to solve classification problems in machine learning. It has so many extensions and variations as follows:

1. **Quadratic Discriminant Analysis (QDA):** For multiple input variables, each class deploys its own estimate of variance.
2. **Flexible Discriminant Analysis (FDA):** it is used when there are non-linear groups of inputs are used, such as splines.
3. **Flexible Discriminant Analysis (FDA):** This uses regularization in the estimate of the variance (actually covariance) and hence moderates the influence of different variables on LDA.

Real-world Applications of LDA

Some of the common real-world applications of Linear discriminant Analysis are given below:

- **Face Recognition**

Face recognition is the popular application of computer vision, where each face is represented as the combination of a number of pixel values. In this case, LDA is used to minimize the number of features to a manageable number before going through the classification process. It generates a new template in which each dimension consists of a

linear combination of pixel values. If a linear combination is generated using Fisher's linear discriminant, then it is called Fisher's face.

- **Medical**

In the medical field, LDA has a great application in classifying the patient disease on the basis of various parameters of patient health and the medical treatment which is going on. On such parameters, it classifies disease as mild, moderate, or severe. This classification helps the doctors in either increasing or decreasing the pace of the treatment.

- **Customer Identification**

In customer identification, LDA is currently being applied. It means with the help of LDA; we can easily identify and select the features that can specify the group of customers who are likely to purchase a specific product in a shopping mall. This can be helpful when we want to identify a group of customers who mostly purchase a product in a shopping mall.

- **For Predictions**

LDA can also be used for making predictions and so in decision making. For example, "will you buy this product" will give a predicted result of either one or two possible classes as a buying or not.

- **In learning**

nowadays, robots are being trained for learning and talking to simulate human work, and it can also be considered a classification problem. In this case, LDA builds similar groups on the basis of different parameters, including pitches, frequencies, sound, tunes, etc.

Difference between Linear Discriminant Analysis and PCA

Below are some basic differences between LDA and PCA:

- PCA is an unsupervised algorithm that does not care about classes and labels and only aims to find the principal components to maximize the variance in the given dataset. At the same time, LDA is a supervised algorithm that aims to find the linear discriminants to represent the axes that maximize separation between different classes of data.
- LDA is much more suitable for multi-class classification tasks compared to PCA. However, PCA is assumed to be an as good performer for a comparatively small sample size.

- Both LDA and PCA are used as dimensionality reduction techniques, where PCA is first followed by LDA.

How to Prepare Data for LDA

Below are some suggestions that one should always consider while preparing the data to build the LDA model:

- **Classification Problems:** LDA is mainly applied for classification problems to classify the categorical output variable. It is suitable for both binary and multi-class classification problems.
- **Gaussian Distribution:** The standard LDA model applies the Gaussian Distribution of the input variables. One should review the univariate distribution of each attribute and transform them into more Gaussian-looking distributions. For e.g., use log and root for exponential distributions and Box-Cox for skewed distributions.
- **Remove Outliers:** It is good to firstly remove the outliers from your data because these outliers can skew the basic statistics used to separate classes in LDA, such as the mean and the standard deviation.
- **Same Variance:** As LDA always assumes that all the input variables have the same variance, hence it is always a better way to firstly standardize the data before implementing an LDA model. By this, the Mean will be 0, and it will have a standard deviation of 1.

Decision Tree Classification Algorithm

- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome**.
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In order to build a tree, we use the **CART algorithm**, which stands for **Classification and Regression Tree algorithm**.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.
- Below diagram explains the general structure of a decision tree:

Note: A decision tree can contain categorical data (YES/NO) as well as numeric data.

Why use Decision Trees?

There are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model. Below are the two reasons for using the Decision tree:

- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
- The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

- **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.
- **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.
- **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- **Branch/Sub Tree:** A tree formed by splitting the tree.
- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.
- **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

How does the Decision Tree algorithm Work?

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into

one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:

Attribute Selection Measures

While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM**. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:

- **Information Gain**
- **Gini Index**

1. Information Gain:

- Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

Information Gain= Entropy(S)- [(Weighted Avg) *Entropy(each feature)]

Entropy: Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:

$$\text{Entropy}(s) = -P(\text{yes}) \log_2 P(\text{yes}) - P(\text{no}) \log_2 P(\text{no})$$

Where,

- **S= Total number of samples**
- **P(yes)= probability of yes**
- **P(no)= probability of no**

2. Gini Index:

- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula:

$$\text{Gini Index} = 1 - \sum_j P_j^2$$

Pruning: Getting an Optimal Decision tree

Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.

A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset. Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as Pruning. There are mainly two types of tree **pruning** technology used:

- **Cost Complexity Pruning**
- **Reduced Error Pruning.**

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm.**
- For more class labels, the computational complexity of the decision tree may increase.

t "**user_data.csv**," which we have used in previous classification models. By using the same dataset, we can compare the Decision tree classifier with other classification models such as [KNN](#), [SVM](#), [LogisticRegression](#), etc.

Steps will also remain the same, which are given below:

- **Data Pre-processing step**
- **Fitting a Decision-Tree algorithm to the Training set**
- **Predicting the test result**
- **Test accuracy of the result(Creation of Confusion matrix)**
- **Visualizing the test set result.**

Generative models are a class of statistical models that generate new data instances. These models are used in unsupervised machine learning to perform tasks.

Probabilistic Generative Model

- Generative models are a class of statistical models that generate new data instances. These models are used in unsupervised machine learning to perform tasks such as probability and likelihood estimation, modelling data points, and distinguishing between classes using these probabilities.
- Generative models rely on the Bayes theorem to find the joint probability. Generative models describe how data is generated using probabilistic models. They predict $P(y | x)$, the probability of y given x , calculating the $P(x,y)$, the probability of x and y .

Naive Bayes

- Naive Bayes classifiers are a family of simple probabilistic classifiers based on applying Bayes' theorem with strong independence assumptions between the features. It is highly scalable, requiring a number of parameters linear in the number of variables (features/predictors) in a learning problem.
- A Naive Bayes Classifier is a program which predicts a class value given a set of attributes.
- For each known class value,

1. Calculate probabilities for each attribute, conditional on the class value.
 2. Use the product rule to obtain a joint conditional probability for the attributes.
 3. Use Bayes rule to derive conditional probabilities for the class variable.
- Once this has been done for all class values, output the class with the highest probability.
 - Naive bayes simplifies the calculation of probabilities by assuming that the probability of each attribute belonging to a given class value is independent of all other attributes. This is a strong assumption but results in a fast and effective method.
 - The probability of a class value given a value of an attribute is called the conditional probability. By multiplying the conditional probabilities together for each attribute for a given class value, we have a probability of a data instance belonging to that class.

Conditional Probability

- Let A and B be two events such that $P(A) > 0$. We denote $P(B|A)$ the probability of B given that A has occurred. Since A is known to have occurred, it becomes the new sample space replacing the original S. From this, the definition is,

$$P(B/A) = P(A \cap B)/P(A)$$

OR

$$P(A \cap B) = P(A) P(B/A)$$

- The notation $P(B | A)$ is read "the probability of event B given event A". It is the probability of an event B given the occurrence of the event A.
- We say that, the probability that both A and B occur is equal to the probability that A occurs times the probability that B occurs given that A has occurred. We call $P(B | A)$ the conditional probability of B given A, i.e., the probability that B will occur given that A has occurred.
- Similarly, the conditional probability of an event A, given B by,

$$P(A/B) = P(A \cap B)/P(B)$$

- The probability $P(A | B)$ simply reflects the fact that the probability of an event A may depend on a second event B. If A and B are mutually exclusive $A \cap B = \emptyset$ and $P(A | B) = 0$.
- Another way to look at the conditional probability formula is :

$$P(\text{Second/First}) = P(\text{First choice and second choice})/P(\text{First choice})$$

- Conditional probability is a defined quantity and cannot be proven.
- The key to solving conditional probability problems is to:

1. Define the events.
2. Express the given information and question in probability notation.
3. Apply the formula.

Joint Probability

- A joint probability is a probability that measures the likelihood that two or more events will happen concurrently.
- If there are two independent events A and B, the probability that A and B will occur is found by multiplying the two probabilities. Thus for two events A and B, the special rule of multiplication shown symbolically is :

$$P(A \text{ and } B) = P(A) P(B).$$

- The general rule of multiplication is used to find the joint probability that two events will occur. Symbolically, the general rule of multiplication is,

$$P(A \text{ and } B) = P(A) P(B | A).$$

- The probability $P(A \cap B)$ is called the joint probability for two events A and B which intersect in the sample space. Venn diagram will readily shows that

$$P(A \cap B) = P(A) + P(B) - P(A \cup B)$$

Equivalently:

$$P(A \cap B) = P(A) + P(B) - P(A \cup B) \leq P(A) + P(B)$$

- The probability of the union of two events never exceeds the sum of the event probabilities.
- A tree diagram is very useful for portraying conditional and joint probabilities. A tree diagram portrays outcomes that are mutually exclusive.

Bayes Theorem

- Bayes' theorem is a method to revise the probability of an event given additional information. Bayes's theorem calculates a conditional probability called a posterior or revised probability.

- Bayes' theorem is a result in probability theory that relates conditional probabilities. If A and B denote two events, $P(A | B)$ denotes the conditional probability of A occurring, given that B occurs. The two conditional probabilities $P(A | B)$ and $P(B | A)$ are in general different.
- Bayes theorem gives a relation between $P(A | B)$ and $P(B | A)$. An important application of Bayes' theorem is that it gives a rule how to update or revise the strengths of evidence-based beliefs in light of new evidence a posterior.
- A prior probability is an initial probability value originally obtained before any additional information is obtained.
- A posterior probability is a probability value that has been revised by using additional information that is later obtained.
- Suppose that $B_1, B_2, B_3 \dots B_n$ partition the outcomes of an experiment and that A is another event. For any number, k, with $1 \leq k \leq n$, we have the formula:

$$P(B_k/A) = \frac{P(A/B_k) \cdot P(B_k)}{\sum_{i=1}^n P(A/B_i) \cdot P(B_i)}$$

Difference between Generative and Discriminative Models

Generative model	Discriminative models
Generative models can generate new data instances.	Discriminative models discriminate between different kinds of data instances
Generative model revolves around the distribution of a dataset to return a probability for a given example.	Discriminative model makes predictions based on conditional probability and is either used for classification or regression.
Generative models capture the joint probability $p(X, Y)$, or just $p(X)$ if there are no labels.	Discriminative models capture the conditional probability $p(Y X)$.
A generative model includes the distribution of the data itself, and tells you how likely a given example is.	A discriminative model ignores the question of whether a given instance is likely, and just tells you how likely a label is to apply to the instance.
Generative models are used in unsupervised machine learning to perform tasks such as probability and likelihood estimation	The discriminative model is used particularly for supervised machine learning.
Example : Gaussians, Naïve Bayes	Example : Logistic regression, SVMs

K-nearest neighbor definition

kNN, or the k-nearest neighbor algorithm, is a machine learning algorithm that uses proximity to compare one data point with a set of data it was trained on and has memorized to make predictions. This instance-based learning affords kNN the 'lazy learning' denomination and enables the algorithm to perform classification or regression problems. kNN works off the assumption that similar points can be found near one another — birds of a feather flock together.

As a classification algorithm, kNN assigns a new data point to the majority set within its neighbors. As a regression algorithm, kNN makes a prediction based on the average of the values closest to the query point.

kNN is a [supervised learning algorithm](#) in which 'k' represents the number of nearest neighbors considered in the classification or regression problem, and 'NN' stands for the nearest neighbors to the number chosen for k.

Brief history of the kNN algorithm

kNN was first developed by Evelyn Fix and Joseph Hodges in 1951 in the context of research performed for the US military¹. They published a paper explaining discriminant analysis, which is a non-parametric classification method. In 1967, Thomas Cover and Peter Hart expanded on the non-parametric classification method and published their "Nearest Neighbor Pattern Classification" paper². Almost 20 years later, the algorithm was refined by James Keller, who developed a "fuzzy KNN" that produces lower error rates³.

Today, the kNN algorithm is the most widely used algorithm due to its adaptability to most fields — from genetics to finance and customer service.

How does kNN work?

The kNN algorithm works as a supervised learning algorithm, meaning it is fed training datasets it memorizes. It relies on this labeled input data to learn a function that produces an appropriate output when given new unlabeled data.

This enables the algorithm to solve classification or regression problems. While kNN's computation occurs during a query and not during a training phase, it has important data storage requirements and is therefore heavily reliant on memory.

For classification problems, the KNN algorithm will assign a class label based on a majority, meaning that it will use the label that is most frequently present around a given data point. In other words, the output of a classification problem is the **mode** of the nearest neighbors.

A distinction: majority voting vs. plurality voting

Majority voting denotes anything over 50% as the majority. This applies if there are two class labels in consideration. However, plurality voting applies if multiple class labels are being considered. In these cases, anything over 33.3% would be sufficient to denote a majority, and hence provide a prediction. Plurality voting is therefore the more accurate term to define kNN's mode.

If we were to illustrate this distinction:

A binary prediction

Y: ☐☐☐☒☐☒☐☒☐☒☐☒☐

Majority vote: ☒☐

Plurality vote: ☒☐

A multi-class setting

Y: ☐☐☐☐☐☐☐☐☐☐

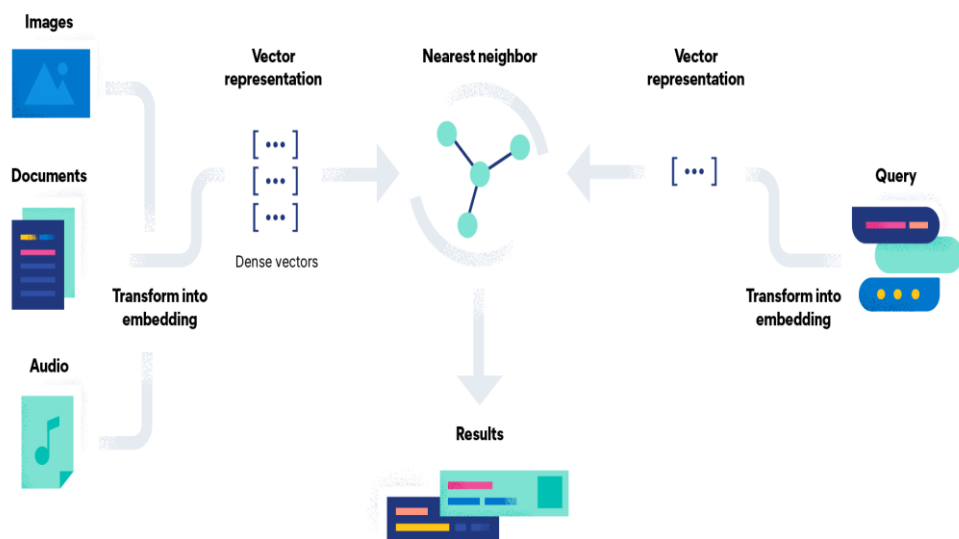
Majority vote: None

Plurality vote: ☐

Regression problems use the **mean** of the nearest neighbors to predict a classification. A regression problem will produce real numbers as the query output.

For example, if you were making a chart to predict someone's weight based on their height, the values denoting height would be independent, while the values for weight would be dependent. By performing a calculation of the average height-to-weight ratio, you could estimate someone's weight (the dependent variable) based on their height (the independent variable).

4 types of computing kNN distance metrics



The key to the kNN algorithm is determining the distance between the query point and the other data points. Determining distance metrics enables decision boundaries. These boundaries create different data point regions. There are different methods used to calculate distance:

Euclidean distance is the most common distance measure, which measures a straight line between the query point and the other point being measured.

Manhattan distance is also a popular distance measure, which measures the absolute value between two points. It is represented on a grid, and often referred to as taxicab geometry — how do you travel from point A (your query point) to point B (the point being measured)?

Minkowski distance is a generalization of Euclidean and Manhattan distance metrics, which enables the creation of other distance metrics. It is calculated in a normed vector space. In the Minkowski distance, p is the parameter that defines the type of distance used in the calculation. If $p=1$, then the Manhattan distance is used. If $p=2$, then the Euclidean distance is used.

Hamming distance, also referred to as the overlap metric, is a technique used with Boolean or string vectors to identify where vectors do not match. In other words, it measures the distance between two strings of equal length. It is especially useful for error detection and error correction codes.

How to choose the best k value

To choose the best k value — the number of nearest neighbors considered — you must experiment with a few values to find the k value that generates the most accurate predictions with the fewest number of errors. Determining the best value is a balancing act:

Low k values make predictions unstable.

Take this example: a query point is surrounded by 2 green dots and one red triangle. If $k=1$ and it happens that the point closest to the query point is one of the green dots, the algorithm will incorrectly predict a green dot as the outcome of the query. Low k values are high variance (the model fits too closely to the training data), high complexity, and low bias (the model is complex enough to fit the training data well).

High k values are noisy.

A higher k value will increase the accuracy of predictions because there are more numbers of which to calculate the modes or means. However, if the k value is too high, it will likely result in low variance, low complexity, and high bias (the model is NOT complex enough to fit the training data well).

Ideally, you want to find a k value that is between high variance and high bias. It is also recommended to choose an odd number for k to avoid ties in classification analysis.

The right k value is also relative to your data set. To choose that value, you might try to find the square root of N , where N is the number of data points in the training dataset. Cross-validation tactics can also help you choose the k value best suited to your dataset.

Advantages of the kNN algorithm

The kNN algorithm is often described as the “simplest” supervised learning algorithm, which leads to its several advantages:

Simple: kNN is easy to implement because of how simple and accurate it is. As such, it is often one of the first classifiers that a data scientist will learn.

Adaptable: As soon as new training samples are added to its dataset, the kNN algorithm adjusts its predictions to include the new training data.

Easily programmable: kNN requires only a few hyperparameters — a k value and a distance metric. This makes it a fairly uncomplicated algorithm.

In addition, the kNN algorithm requires no training time because it stores training data and its computational power is used only when making predictions.

Challenges and limitations of kNN

While the kNN algorithm is simple, it also has a set of challenges and limitations, due in part to its simplicity:

Difficult to scale: Because kNN takes up a lot of memory and data storage, it brings up the expenses associated with storage. This reliance on memory also means that the algorithm is computationally intensive, which is in turn resource-intensive.

Curse of dimensionality: This refers to a phenomenon that occurs in computer science, wherein a fixed set of training examples is challenged by an increasing number of dimensions and the inherent increase of feature values in these dimensions. In other words, the model's training data can't keep up with the evolving dimensionality of the hyperspace. This means that predictions become less accurate because the distance between the query point and similar points grows wider — on other dimensions.

Overfitting: The value of k , as shown earlier, will impact the algorithm's behavior. This can happen especially when the value of k is too low. Lower values of k can overfit the data, while higher values of k will 'smooth' the prediction values because the algorithm averages values over a greater area.

Top kNN use cases

The kNN algorithm, popular for its simplicity and accuracy, has a variety of applications, especially when used for classification analysis.

Relevance ranking: kNN uses [natural language processing \(NLP\)](#) algorithms to determine which results are most relevant to a query.

Similarity search for images or videos: Image similarity search uses natural language descriptions to find images matching from text queries.

Pattern recognition: kNN can be used to identify patterns in text or digit classification.

Finance: In the financial sector, kNN can be used for stock market forecasting, currency exchange rates, etc.

Product recommendations and recommendation engines: Think Netflix! "If you liked this, we think you'll also like..." Any site that uses a version of that sentence, overtly or not, is likely using a kNN algorithm to power its recommendation engine.

Healthcare: In the field of medicine and medical research, the kNN algorithm can be used in genetics to calculate the probability of certain gene expressions. This allows doctors to predict the likelihood of cancer, heart attacks, or any other hereditary conditions.

Data preprocessing: The kNN algorithm can be used to estimate missing values in datasets.